Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251176**
Nama Lengkap	**David Alexsi Cahyadi**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Definisi Perulangan

Dalam pemrograman, jalannya suatu program dapat diatur menggunakan beberapa bentuk struktur kontrol, yaitu **sekuensial**, **percabangan**, dan **perulangan**, ataupun gabungan dari ketiganya. Struktur kontrol tersebut berfungsi untuk menentukan bagaimana suatu program dijalankan. Salah satu bentuk struktur kontrol yang sangat sering digunakan adalah **perulangan**.

Perulangan digunakan ketika suatu program perlu melakukan proses yang sama secara berulang-ulang. Misalnya ketika sebuah program harus menjalankan suatu perintah berkali-kali, memproses data secara bertahap, atau mengakses banyak data dalam sebuah struktur seperti **list**, **tuple**, **queue**, atau **stack**. Dengan adanya perulangan, programmer tidak perlu menulis perintah yang sama berkali-kali secara manual.

Dalam bahasa pemrograman Python, perulangan dapat dilakukan menggunakan beberapa cara, yaitu dengan **perulangan for**, **perulangan while**, maupun dengan teknik **rekursif**. Pada materi ini, fokus utama pembahasan adalah penggunaan perulangan **for** dan **while**, karena kedua bentuk ini paling sering digunakan dalam pembuatan program.

Bentuk Perulangan for

Perulangan **for** biasanya digunakan ketika jumlah perulangan sudah diketahui sejak awal. Contohnya adalah ketika program harus membaca sejumlah file tertentu, atau ketika

melakukan operasi terhadap suatu rentang nilai tertentu. Dalam kondisi seperti ini, jumlah perulangan dapat ditentukan terlebih dahulu sebelum program dijalankan.

Dalam Python, perulangan **for** sering digunakan bersama **fungsi range()**. Fungsi ini untuk menghasilkan rentang angka yang akan digunakan sebagai dasar perulangan. Bentuk umum penggunaan fungsi range sebagai berikut:

- **range(stop)** menghasilkan angka 0 sampai stop-1.
- **range(start, stop, step)** menghasilkan angka dari start sampai stop-1 dengan kenaikan sesuai nilai step.

Sebagai contoh, jika ingin menampilkan bilangan 1 sampai 100, dapat digunakan perulangan for yang memanfaatkan fungsi range(). Variabel yang digunakan pada perulangan ini biasanya disebut **counter**, yaitu variabel yang nilainya berubah pada setiap iterasi.

Selain itu, jika nilai counter tidak diperlukan didalam perulangan, Python memungkinkan penggunaan variabel underscore/garis bawah (_) sebagai penanda bahwa nilai tersebut tidak digunakan.

Perulangan **for** juga dapat menggunakan **step negatif**. Hal ini digunakan jika perulangan berjalan dari nilai yang besar ke nilai yang lebih kecil. Sebagai contoh, untuk menampilkan bilangan genap 100 sampai 2, nilai step dapat dibuat -2, sehingga nilai akan berkurang pada setiap iterasi.

Bentuk Perulangan While

Berbeda dengan perulangan **for**, perulangan **while** biasanya digunakan ketika jumlah perulangan belum diketahui sebelumnya. Perulangan ini akan terus dijalankan selama kondisi yang diberikan masih bernilai **True**.

Bentuk umum perulangan while adalah menjalankan suatu operasi selama kondisi tertentu masih terpenuhi. Jika kondisi tersebut berubah menjadi **False**, maka perulangan akan berhenti.

Sebagai contoh, perulangan while dapat digunakan untuk meminta input dari pengguna hingga pengguna memasukkan data yang sesuai dengan ketentuan program. Misalnya program meminta pengguna untuk memasukkan **bilangan genap**. Jika pengguna memasukkan bilangan ganjil, maka program akan terus meminta input sampai pengguna memasukkan bilangan genap. Seperti pada gambar di bawah ini (Gambar 6.1).

```
1  bilangan = 0
2  genap = False
3  while genap == False:
4      bilangan = int(input('Masukkan bilangan genap: '))
5      if bilangan % 2 == 0:
6          genap = True
7  print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Gambar 6.1: Contoh program bilangan genap/tidak, Sumber: https://shorturl.at/5NIQl

Pada contoh tersebut, jumlah perulangan tidak dapat diketahui sejak awal karena tergantung pada berapa kali pengguna memasukkan nilai yang salah. Oleh karena itu, penggunaan **while** lebih tepat dibandingkan menggunakan **for**.

```
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
Masukkan bilangan genap: 99
Masukkan bilangan genap: 23
Masukkan bilangan genap: 11
Masukkan bilangan genap: 37
Masukkan bilangan genap: 87651
Masukkan bilangan genap: 88
88 yang anda masukkan adalah bilangan genap
```

Gambar 6.2: Penggunaan while untuk mengambil input dari pengguna sampai sesuai dengan permintaan, Sumber: https://shorturl.at/5NIQl

Penggunaan Break dan Continue

Dalam perulangan, terkadang diperlukan mekanisme untuk mengontrol jalannya iterasi. Python menyediakan dua perintah yang dapat digunakan untuk tujuan tersebut, yaitu **break** dan **continue**.

Perintah **break** digunakan untuk menghentikan perulangan secara langsung, meskipun kondisi perulangan sebenarnya masih terpenuhi. Ketika program menjalankan perintah break, perulangan akan langsung berhenti dan program akan melanjutkan eksekusi ke bagian setelah perulangan tersebut.

Sebagai contoh, jika sebuah perulangan menampilkan angka dari 1 sampai 10, tetapi program ingin menghentikan proses ketika angka mencapai 5, maka perintah break dapat digunakan untuk menghentikan perulangan pada saat kondisi tersebut tercapai.

```python
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
```

Gambar 6.3: Contoh program bilangan genap/tidak, Sumber: https://shorturl.at/5NIQl

Sementara itu, perintah **continue** digunakan untuk melewati satu iterasi perulangan, kemudian langsung melanjutkan ke iterasi berikutnya. Artinya, bagian kode setelah continue pada iterasi tersebut tidak akan dijalankan.

Contoh penggunaan continue adalah ketika program menampilkan angka dari 1 sampai 10, tetapi ingin melewati angka 6. Pada saat nilai perulangan sama dengan 6, program akan menjalankan continue sehingga angka tersebut tidak ditampilkan dan perulangan langsung berlanjut ke angka berikutnya.

```
for i in range(1, 11):
    if i == 6:
        continue
    else:
        print(i)
print('Selesai')
```

Gambar 6.4: Contoh penggunaan continue, Sumber: https://shorturl.at/5NIQl

```
C:\ProgramData\Microsoft\Win
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
1
2
3
4
5
7
8
9
10
Selesai
```

Gambar 6.5: Output penggunaan continue untuk melewatkan iterasi saat counter(i) bernilai 6, Sumber: https://shorturl.at/5NIQl

Konversi dari Bentuk for Menjadi Bentuk while

Pada banyak kasus, perulangan **for** dapat diubah menjadi perulangan **while**. Hal ini karena kedua bentuk perulangan tersebut memiliki komponen dasar yang sama, yaitu:

1. **Nilai awal (initial value)** untuk perulangan

2. **Kondisi akhir (termination condition)** yang menentukan kapan perulangan berhenti.
3. **Langkah perubahan nilai (step)** yang menyebabkan iterasi bergerak menuju kondisi akhir.

Sebagai contoh perulangan for yang menampilkan angka dari 1 sampai 10 sebenarnya memiliki nilai awal 1, nilai akhir 10, dan langkah perubahan +1 pada setiap iterasi. Jika perulangan tersebut dikonversi ke dalam bentuk while, maka programmer harus secara eksplisit menuliskan nilai awal variabel, kondisi perulangan, serta perubahan nilai variabel pada setiap iterasi.

```
for i in range(1, 11):
    print(i)
```

Gambar 6.6: Contoh perulangan for, Sumber: https://shorturl.at/5NIQl

Perulangan **while** lebih fleksibel dalam proses iterasi, karena programmer harus menentukan secara, manual bagaimana variabel perulangan berubah pada setiap langkah.

```
i = 1                //awal
while i <= 10:       //kondisi akhir
    print(i)
    i = i + 1        //step
```

Gambar 6.7: Contoh perulangan while, Sumber: https://shorturl.at/5NIQl

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github:https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 6.1

Soal:

Latihan 6.1 Buatlah program yang menerapkan perhitungan perkalian dengan menggunakan penjumlahan. Buatlah fungsi perkalian() dalam program tersebut! Berikut ini adalah beberapa contoh perhitungan yang diharapkan:

- 6x5=5+5+5+5+5+5=30.
- 7x10=10+10+10+10+10+10+10=70

Source Code:

```
#Latihan 6.1
def perkalian(a,b):
    hasil = 0
    proses = ""

    for i in range(a):
        hasil = hasil + b
        proses += str(b)
        if i < a - 1:
            proses += " + "

    print(f"{a} x {b} = {proses} = {hasil}")


a = int(input("Masukkan angka pertama: "))
b = int(input("Masukkan angka kedua: "))

perkalian(a,b)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\dvdal> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 6/soal01.py"
Masukkan angka pertama: 10
Masukkan angka kedua: 2
10 x 2 = 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 = 20
PS C:\Users\dvdal> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 6/soal01.py"
Masukkan angka pertama: 2
Masukkan angka kedua: 10
2 x 10 = 10 + 10 = 20
```

Penjelasan:

Di dalam fungsi **perkalian(a, b)** dibuat dua variabel, yaitu **hasil** yang diinisialisasi dengan nilai 0 untuk menyimpan hasil penjumlahan, serta **proses** yang berupa string kosong untuk menyimpan tampilan proses perhitungan. Variabel **proses** nantinya akan digunakan untuk menampilkan bentuk penjumlahan seperti **5 + 5 + 5 + 5 + 5 + 5**.

Lalu, program akan menggunakan perulangan **for i in range(a)** yang akan berjalan sebanyak nilai **a**. Pada setiap perulangan, nilai **b** ditambahkan ke variabel **hasil** menggunakan operasi **hasil = hasil + b**. Dengan cara ini, proses perkalian dilakukan melalui penjumlahan yang dilakukan secara berulang.

Selain menghitung hasil penjumlahan, program juga menyusun tampilan proses perhitungannya dengan menambahkan nilai **b** ke variabel **proses**. Program juga menambahkan tanda " + " di antara angka menggunakan kondisi **if < a – 1** agar tanda tambah tidak muncul di bagian akhir.

Setelah perulangan selesai, program akan menampilkan hasil dalam bentuk perkalian, proses penjumlahan, dan akhirnya menggunakan fungsi **print**. Sebelum fungsi dijalankan, program terlebih dahulu meminta pengguna memasukkan dua buah bilangan melalui **input()**, yang kemudian disimpan dalam variabel **a** dan **b**. Nilai tersebut kemudian dikirim sebagai argument ke fungsi **perkalian(a,b)**. Sebagai contoh, ketika pengguna memasukkan

nilai **10** dan **2**, maka program akan menghasilkan output 10 x 2 = 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 + 2 = 20.

Latihan 6.2

Soal:

Latihan 6.2 Buatlah program yang dapat menampilkan deret bilangan ganjil dari batas bawah dan batas atas yang diberikan oleh pengguna. Jika ternyata batas atas < batas bawah, berarti deret tersebut dimulai dari batas atas, sampai batas bawah (negatif range). Buatlah fungsi ganjil() dalam program tersebut! Berikut ini adalah contoh hasil yang diharapkan:

- bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29.

- bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83.

Source Code:

```
#Latihan 6.2
def ganjil(bawah, atas):
    if bawah < atas:
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah < atas, berarti dari
kecil ke besar, maka hasilnya adalah:", end=" ")
        nentu = True
        for i in range(bawah, atas + 1):
            if i % 2 != 0:
                if not nentu:
                    print(", ", end="")
                print(i, end="")
                nentu = False

    else:
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah > atas, berarti dari
besar ke kecil, maka hasilnya adalah:", end=" ")
        nentu = True
        for i in range(bawah, atas -1, -1):
            if i % 2 != 0:
                if not nentu:
```

```
                    print(", ", end="")
                print(i, end='')
                nentu = False

bawah = int(input("Masukkan batas bawah: "))
atas = int(input("Masukkan batas atas: "))

ganjil(bawah,atas)
```

Output:

```
Masukkan batas bawah: 10
Masukkan batas atas: 30
bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29
```

Dari kecil ke besar

```
Masukkan batas bawah: 97
Masukkan batas atas: 82
bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83
```

Dari besar ke kecil

Penjelasan:

Pada awal program, pengguna diminta untuk memasukkan nilai batas bawah dan batas atas menggunakan fungsi **input()**. Nilai yang dimasukkan kemudian dikonversi menjadi tipe data integer agar dapat digunakan dalam proses perhitungan. Setelah itu, kedua nilai tersebut dikirim sebagai parameter ketika fungsi **ganjil()** dipanggil.

Di dalam fungsi **ganjil**, program terlebih dahulu memeriksa kondisi apakah bawah lebih kecil dari atas atau tidak. Jika **bawah < atas**, maka deret bilangan akan ditampilkan dari nilai kecil ke besar menggunakan perulangan **for i in range(bawah, atas + 1)**. Sebaliknya, jika **bawah > atas**, maka deret akan ditampilkan dari nilai besar ke nilai kecil menggunakan perulangan **for i in range(bawah, atas -1, -1)**.

Selama proses perulangan berlangsung, program akan memeriksa apakah suatu bilangan merupakan bilangan ganjil atau bukan, dengan menggunakan operasi modulus **i % 2 != 0**. Jika kondisi tersebut bernilai benar, maka bilangan tersebut akan ditampilkan sebagai bagian dari deret bilangan ganjil.

Agar tampilan deret lebih rapi, program menggunakan variabel **nentu** untuk mengatur penambahan tanda koma di antara bilangan. Variabel ini digunakan untuk memastikan bahwa tanda koma hanya muncul di antara angka, dan tidak muncul di awal deret.

Latihan 6.3

Soal:

Latihan 6.3 Buatlah sebuah program penghitung nilai Indeks Prestasi Semester (IPS). Input bagi program:

- Jumlah mata kuliah
- Nilai A, B, C, dan D untuk setiap mata kuliah mahasiswa. Diasumsikan sks setiap mata kuliah selalu 3. Kemudian bobot dari masing-masing nilai adalah: A=4, B=3, C=2, D=1.

Output program ialah hasil IPS yang didapatkan. Jalannya program seperti pada Gambar 6.7. Tips: Gunakan kontrol percabangan di dalam perulangan.

Source Code:

```
#Latihan 6.3
matkullu = int(input("Berapa jumlah mata kuliah? "))
bobot = 0
sks = 3
tb = 0

for i in range(1, matkullu +1):
    nilai =  input(f"Nilai MK {i}: ").upper()
    if nilai == "A":
        bobot = 4
    elif nilai == "B":
```

```
        bobot = 3
    elif nilai == "C":
        bobot = 2
    elif nilai == "D":
        bobot = 1
    else:
        bobot = 0

    tb += bobot * sks

ts = matkullu * sks
ips = tb / ts

print(f"Nilai IPS anda smester ini {ips:.2f}")
```

Output:

```
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: A
Nilai MK 5: D
Nilai MK 6: C
Nilai IPS anda smester ini 2.67
```

Penjelasan:

Pengguna diminta untuk memasukkan jumlah mata kuliah yang diambil pada semester tersebut. Kemudian program akan menginisialisasi beberapa variabel, yaitu **bobot** untuk menyimpan bobot nilai huruf, **sks** yang bernilai 3 karena setiap mata kuliah diasumsikan memiliki 3 SKS, serta **tb** yang digunakan untuk menyimpan total bobot nilai yang telah dikalikan dengan sks.

Lalu, program menggunakan perulangan **for** untuk meminta input nilai setiap mata kuliah sesuai dengan jumlah mata kuliah yang telah dimasukkan. Nilai yang dimasukkan oleh pengguna kemudian diubah menjadi huruf kapital menggunakan fungsi **.upper()** agar

memudahkan proses pengecekan. Di dalam perulangan tersebut digunakan struktur percabangan **if-elif-else** untuk menentukan bobot nilai berdasarkan huruf yang dimasukkan, yaitu nilai A memiliki bobot 4, nilai B berbobot 3, nilai C berbobot 2, dan D berbobot 1. Jika pengguna memasukkan nilai selain huruf tersebut, maka bobot dianggap bernilai 0.

Setelah bobot nilai ditentukan, program menghitung total bobot nilai dengan cara mengalikan bobot dengan jumlah SKS, kemudian menambahkannya ke variabel **tb**. Setelah semua nilai mata kuliah diproses, program menghitung total SKS dengan mengalikan jumlah mata kuliah dengan SKS per mata kuliah. Setelah itu, IPS dihitung dengan membagi total bobot nilai (**tb**) dengan total SKS (**ts**).